

Feature Selection in Statistical Machine Learning

Garvesh Raskutti

Lecture 15

Recap and outline

- Undirected graphical model selection
- Connection between neighborhood selection and feature selection
- Issues and examples
- Directed graphical models introduction

Undirected graphical models

Undirected graphical models:

$$G = (V, E)$$

where $V = (X_1, X_2, \dots, X_p)$, E undirected edges

Nice and simple relationship:

$$(X_j, X_k) \in E \Leftrightarrow X_j \not\perp\!\!\!\perp X_k | X_S$$

where $S = \{1, 2, \dots, p\} / \{j, k\}$

In words: There exists an edge $(X_j, X_k) \in E$ if and only if you condition on all other nodes/random variables, X_j and X_k are dependent

Problem/goal of interest: Learning graphical model from data

Given observed feature data:

$$\{X_j^{(i)}\}_{j=1,2,\dots,p}^{i=1,2,\dots,n}$$

Problem: Can we learn/estimate the undirected graphical model from observed data?

$$\mathcal{A} : \{X_j^{(i)}\}_{j=1,2,\dots,p}^{i=1,2,\dots,n} \Rightarrow \widehat{G} = (V, \widehat{E})$$

where $V = (X_1, X_2, \dots, X_p)$

Undirected graphical models and feature selection

For feature selection:

$$X_j \in \tilde{S} \Leftrightarrow Y \not\perp X_j | X_{-j}$$

For undirected graphical models:

$$(X_k, X_\ell) \in E \Leftrightarrow X_k \not\perp X_\ell | X_S$$

where $S = \{1, 2, \dots, p\} / \{k, \ell\}$.

Question: Is there a connection between whether there is a feature X_j in $\tilde{S}$ or there is an edge between $(X_k, X_\ell) \in E$?

Neighborhood selection as feature selection

For neighborhood selection of X_k in undirected graphical model:

$$\mathcal{N}(X_k) = \{\ell : \ell \in \{1, 2, \dots, p\} \setminus \{k\}, X_k \not\perp\!\!\!\perp X_\ell | X_S \text{ where } S = \{1, 2, \dots, p\} \setminus \{k, \ell\}\}$$

For feature selection of Y recall that:

$$\tilde{S} = \{j : j \in \{1, 2, \dots, p\}, Y \not\perp\!\!\!\perp X_j | X_{-j}\}$$

Therefore:

Neighborhood selection for X_k is equivalent to feature selection where X_k represents Y and the remaining features $\{X_\ell : \ell \in \{1, 2, \dots, p\} \setminus \{k\}\}$ is feature set

And, to learn the full graph structure, run neighborhood selection for all p nodes $(X_1, X_2, \dots, X_p)$.

Neighborhood selection code

```
import numpy as np
from sklearn.linear_model import LassoCV
from sklearn.preprocessing import StandardScaler

def neighborhood_selection(X, alpha=0.5):
    n_samples, n_features = X.shape
    adjacency_matrix = np.zeros((n_features, n_features))

    # Standardize data
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    for j in range(n_features):
        # Response is X_j
        y = X_scaled[:, j]
        # Predictors are all other variables
        X_rest = np.delete(X_scaled, j, axis=1)

        # Fit Lasso with cross-validation if alpha is None
        if alpha is None:
            lasso = LassoCV(cv=5, positive=False).fit(X_rest, y)
            coef = lasso.coef_
        else:
            lasso = Lasso(alpha=alpha).fit(X_rest, y)
            coef = lasso.coef_

        # Fill adjacency matrix
        idx = 0
        for k in range(n_features):
            if k == j:
                continue
            if coef[idx] != 0:
                adjacency_matrix[j, k] = 1
            idx += 1

    # Symmetrize adjacency (OR rule)
    adjacency_matrix = np.maximum(adjacency_matrix, adjacency_matrix.T)
    return adjacency_matrix
```

Stock return data example

- Graphical models also useful for stock return data for different S&P 500 companies
- How does stock price of one influence the other given a set of stocks
- For simplicity we consider 10 companies: Apple Microsoft, Google, JPMorgan, CVX, Johnson and Johnson, Coca cola, Pepsi, Visa, Proctor and Gamble
- Can potentially help you determine which stocks to invest in depending on the goal (i.e. diversification, short term reward maximization, e.t.c.)

Stock return loading data

```
import numpy as np
import pandas as pd
import yfinance as yf
from sklearn.linear_model import LassoCV
from sklearn.preprocessing import StandardScaler
import itertools
import matplotlib.pyplot as plt
import networkx as nx

# -----
# 1. Download data
# -----

tickers = [
    "AAPL", "MSFT", "GOOG",
    "JPM", "CVX", "JNJ",
    "PG", "KO", "PEP",
    "V"
]

data = yf.download(
    tickers,
    start="2022-01-01",
    end="2024-01-01",
    auto_adjust=True
) ["Close"]
```

Stock returns neighborhood selection

```
n, p = X.shape
adj_matrix = np.zeros((p, p))

for j in range(p):
    y = X.iloc[:, j].values
    X_minus_j = X.drop(columns=X.columns[j]).values

    model = LassoCV(cv=10).fit(X_minus_j, y)
    coefs = model.coef_

    idx = 0
    for k in range(p):
        if k == j:
            continue
        if abs(coefs[idx]) > 1e-5:
            adj_matrix[j, k] = 1
        idx += 1

# Symmetrize (OR rule)
adj_matrix = np.maximum(adj_matrix, adj_matrix.T)
```

Stock return graph display code

```
G = nx.Graph()

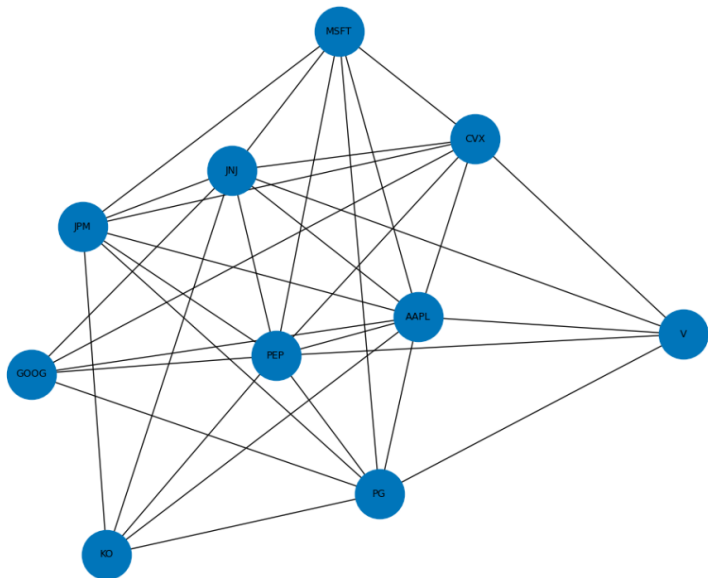
for i in range(p):
    G.add_node(tickers[i])

for i, j in itertools.combinations(range(p), 2):
    if adj_matrix[i, j] == 1:
        G.add_edge(tickers[i], tickers[j])

plt.figure(figsize=(10,8))
pos = nx.spring_layout(G, seed=42)
nx.draw(G, pos, with_labels=True, node_size=2000, font_size=9)
plt.title("Stock Return Graph via Neighborhood Selection")
plt.show()
```

Stock return graph

Stock Return Graph via Neighborhood Selection



Stock return improving interpretability

- How stable is this graph (recall stability for feature selection)?
- Since we are choosing λ via cross-validation which may over-select, what happens if make λ larger to yield a sparser graph?
- How do we make the graph more sparse and stable?

Recall: Stability selection for feature selection

- Run method multiple times over different splits/perturbations of the data
- Determine a feature set for each split of the data
- Assign a probability to a feature being selected by averaging over whether it appears in each split
- Threshold graph based on probability π

Stability selection for learning graphs

```
B = 100
n, p = X.shape

selection_counts = np.zeros((p, p))

for b in range(B):

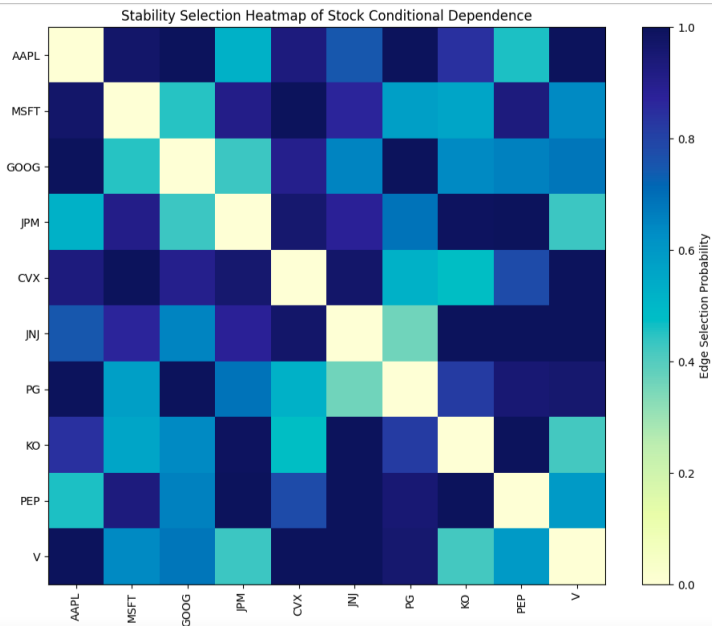
    # subsample 50%
    idx = np.random.choice(n, n//2, replace=False)
    X_sub = X[idx]

    adj = neighborhood_selection(X_sub, alpha=0.02)

    selection_counts += adj

selection_prob = selection_counts / B
prob_threshold = 0.80
adj_matrix = selection_prob >= prob_threshold
```

Stability selection probability matrix



Stability selection graph display

```
G = nx.Graph()

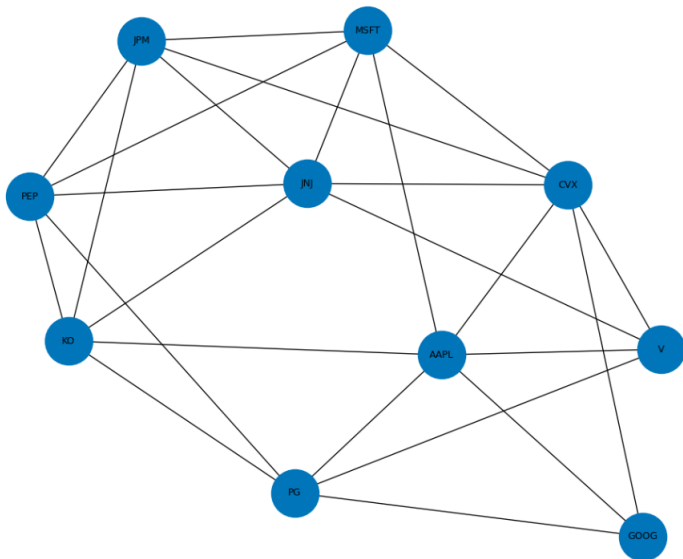
for i in range(p):
    G.add_node(tickers[i])

for i, j in itertools.combinations(range(p), 2):
    if adj_matrix[i, j] == 1:
        G.add_edge(tickers[i], tickers[j])

plt.figure(figsize=(10,8))
pos = nx.spring_layout(G, seed=42)
nx.draw(G, pos, with_labels=True, node_size=2000, font_size=9)
plt.title("Stock Return Graph via Neighborhood Selection")
plt.show()
```

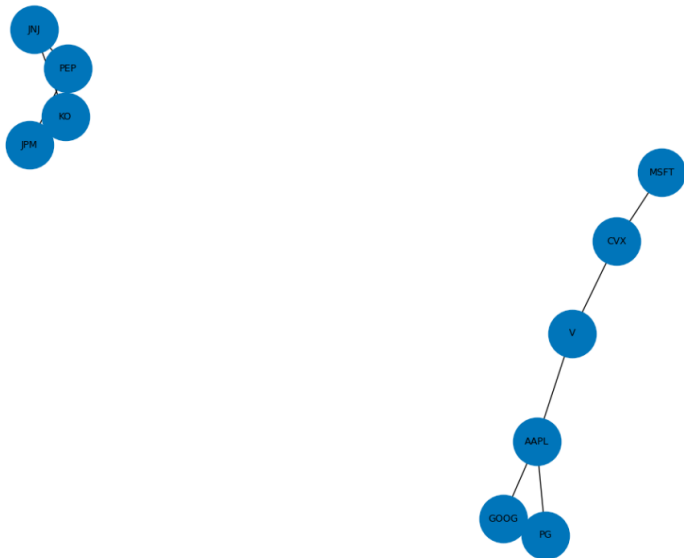
Stability selection for $\pi = 0.8$

Stock Return Graph via Neighborhood Selection



Stability selection for $\pi = 0.99$

Stock Return Graph via Neighborhood Selection



Interpreting stock return graph and open questions

- For $\pi = 0.99$, graph highly interpretable, effectively clustering into different sectors
- Clusters including Coca Cola and Pepsi, Google and Apple connected
- One cluster including tech stocks, the other including Pepsi and Coca Cola

Example: MNIST data pixel value

- We focus on MNIST digit image data in the context of undirected graphical model learning
- Focus on unsupervised 6×6 pixel patches where each pixel is a graph node/feature
- Expect there to be local/spatial structure

Image data: What kind of graph do we expect?

MNIST pre-processing

```
import numpy as np

data = np.load("mnist.npz")

x_train = data["x_train"]
y_train = data["y_train"]

print(x_train.shape)

(60000, 28, 28)

def sample_patches(images, patch_size=6, n_patches=5000):

    patches = []

    for _ in range(n_patches):

        img = images[np.random.randint(len(images))]

        i = np.random.randint(0, 28 - patch_size)
        j = np.random.randint(0, 28 - patch_size)

        patch = img[i:i+patch_size, j:j+patch_size]

        patches.append(patch.flatten())

    return np.array(patches)

X = sample_patches(x_train)

print(X.shape)

(5000, 36)
```

MNIST neighborhood selection code

```
from sklearn.preprocessing import StandardScaler
```

```
X = StandardScaler().fit_transform(X)
```

```
from sklearn.linear_model import Lasso
```

```
def neighborhood_selection(X, alpha=0.2):
```

```
    n, p = X.shape
```

```
    adj = np.zeros((p, p))
```

```
    for j in range(p):
```

```
        y = X[:, j]
```

```
        X_minus = np.delete(X, j, axis=1)
```

```
        model = Lasso(alpha=alpha).fit(X_minus, y)
```

```
        coefs = model.coef_
```

```
        idx = 0
```

```
        for k in range(p):
```

```
            if k == j:
```

```
                continue
```

```
            if abs(coefs[idx]) > 1e-6:
```

```
                adj[j, k] = 1
```

```
            idx += 1
```

```
    return np.maximum(adj, adj.T)
```

```
adj = neighborhood_selection(X)
```


MNIST graph generation code

```
import networkx as nx
import matplotlib.pyplot as plt

p = 36
coords = [(i//6, i%6) for i in range(p)]

G = nx.Graph()

for i in range(p):
    G.add_node(i, pos=coords[i])

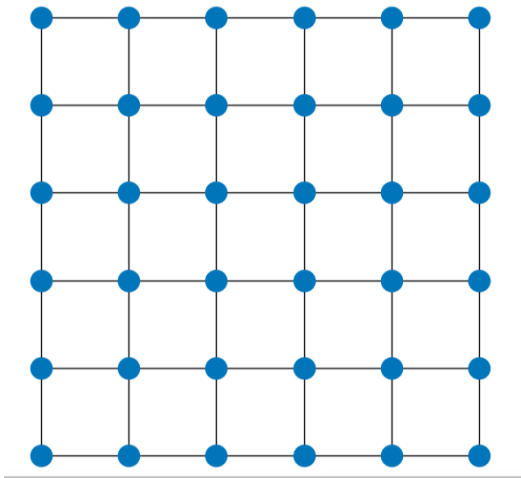
for i in range(p):
    for j in range(i+1, p):
        if adj[i,j] == 1:
            G.add_edge(i,j)

pos = {i:(coords[i][1], -coords[i][0]) for i in range(p)}

plt.figure(figsize=(6,6))
nx.draw(G, pos, node_size=300)
plt.title("Graphical Model of MNIST Pixel Patch")
plt.show()
```

MNIST graph generated

Graphical Model of MNIST Pixel Patch



Interpreting image patch results

- Learned graph perfectly reflects expected spatial dependence
- Undirected graphical models naturally reflect spatial structure
- Proof of concept for neighborhood selection

Summary of undirected graphical models

Undirected graphical models:

$$G = (V, E)$$

where $V = (X_1, X_2, \dots, X_p)$, E undirected edges

Nice and simple relationship:

$$(X_j, X_k) \in E \Leftrightarrow X_j \not\perp\!\!\!\perp X_k | X_S$$

where $S = \{1, 2, \dots, p\} / \{j, k\}$

In words: There exists an edge $(X_j, X_k) \in E$ if and only if you condition on all other nodes/random variables, X_j and X_k are dependent

Directed graphical models

Directed graphical models:

$$G = (V, E)$$

where $V = (X_1, X_2, \dots, X_p)$, E directed edges

$(X_j, X_k) \in E \Leftrightarrow$ What is conditional dependence relationship?

Directed acyclic graphical (DAG) models/Bayesian networks

Directed graphical models:

$$G = (V, E)$$

where $V = (X_1, X_2, \dots, X_p)$, E directed edges

- *Acyclic* means there are no directed cycles/loops in the graph
- DAGs all have a (possibly non-unique) causal ordering

DAG examples